BioAudit

Feature & Function Guide — what the program does, how it works, and what was deliberately left out

> **In one sentence:** BioAudit is a security-testing tool that checks whether an Android app's **biometric authentication** (fingerprint / face unlock) is actually *secure* — not merely whether it *works* — and explains every problem it finds in plain language with a concrete fix.

1. What it is

Black-box security scanner ("Mode B"). It tests an app the way an outside attacker would — from the outside, with no access to the app's source code and no special privileges on the phone. You point it at an installed app or an APK file and it reports weaknesses.

No root required. Everything runs over an ordinary USB-debugging (ADB) connection. You do not have to "jailbreak" or modify the phone, which means it works on normal consumer devices.

Deterministic detection, AI explanation. Whether something is a vulnerability is decided by fixed rules (so results are repeatable and trustworthy). A separate AI layer is used only to *describe* each confirmed problem and suggest a fix — the AI never decides what counts as a vulnerability.

2. How it works — the pipeline

A full assessment runs these stages in order:

1. **Authorization gate** — You must pass `--i-am-authorized` to confirm you own or are allowed to test the app. The tool refuses to probe anything without it.
2. **Static analysis** — Reads the APK's manifest and its compiled code identifiers *without running the app* — spotting risky settings and insecure biometric patterns.
3. **Runtime probing** — Launches the app's exposed components over ADB to see whether protected features can be reached without logging in, and whether they leak secret information through their responses.
4. **Passive observation** — Watches the app's system logs and backup settings for leaked secrets or extractable data.
5. **Scoring & explanation** — Ranks every finding by severity, then the AI layer turns each one into a plain-language explanation plus a concrete fix (with secrets stripped out first).
6. **Output** — Saves the run to a local history database and exports a shareable HTML/PDF report; results can also be browsed in the desktop app or dashboard.

3. Ways to use it

Interface	What it is (plain English)
Command line (CLI)	Type commands in a terminal. Best for automation and quick runs.
Desktop app (PySide6 GUI)	A windowed application with tabs — Scan APK, Assess Device, History — for people who prefer buttons over commands.
Standalone .exe	A single Windows file that bundles everything. Double-click to open the app; no Python install needed.
Streamlit dashboard	A browser-based dashboard for reviewing past test results.

Commands: `scan-apk` (static only, no device) · `assess` (full test on a connected device) · `gui` · `dashboard`.

4. The detectors — what it actually finds

ACTIVE detectors run today in `scan-apk` and/or `assess`.

Manifest analysis ACTIVE

Reads `AndroidManifest.xml`. Flags a **debuggable** build (ships with a developer backdoor), **allowBackup** enabled (private data can be copied off the device), and **exported components** with no permission guard (app screens/services left open to other apps).

Biometric-implementation scan ACTIVE

DEX string-pool identifier scan. Looks inside the app's compiled code for tell-tale names. The key one: **"boolean-only authentication"** — the app trusts a simple "fingerprint OK" yes/no signal without binding it to a cryptographic key, which attackers can bypass. (It reads the code's name table directly rather than fully decompiling, so this is fast even on large apps.)

IPC / exported-component authorization oracle ACTIVE (headline check)

Probes exported components for auth bypass. Asks the sharpest question: can a feature that is *supposed* to sit behind the fingerprint prompt be opened *directly*, skipping login entirely? If the "secret" screen launches without authenticating, the biometric gate is decorative.

Auth-state / response oracle ACTIVE (side-channel test)

Distinguishable-response side channel. A more subtle attack. It sends an exposed component a **valid-looking** identifier and an **obviously-invalid** one and checks whether the app *answers them differently*. If it does, that difference is a leak an attacker can exploit to guess valid usernames or brute-force a token — a classic "oracle" side channel. Unlike timing attacks, this is **deterministic**: a different answer is a hard fact, so it fires reliably every run.

Logcat leakage observer ACTIVE

Scans system logs for secrets. Reads the phone's log stream for tokens, passwords, keys, or "authentication succeeded" messages the app should never have written down.

allowBackup extraction check ACTIVE

Backup-extractability check. If backups are allowed, private app data can be pulled off the device with a standard tool and no root — flagged so the developer turns backups off for sensitive data.

5. Supporting machinery

Component	What it does (plain English)
Severity & recommendation engine	Ranks findings Critical→Info and attaches a concrete fix to each. Lower-confidence static findings are automatically dialled back one step.

AI explanation layer (Gemini)	Turns each confirmed finding into a readable explanation and remediation. It is **grounded** (given a curated knowledge base so it cites real guidance, not guesses) and only ever explains — it never decides what is a vulnerability.
Secret redaction	Strips tokens/keys out of the evidence *before* anything is sent to the AI, so real secrets never leave the machine.
SQLite history	Every run is saved locally so you can compare results over time.
HTML / PDF reports	Each assessment exports a shareable report of all findings.
VulnDemo test app	A deliberately-insecure demo Android app included with the project so every detector can be demonstrated end-to-end against a real device.

6. OWASP Mobile Top 10 coverage

The tool honestly maps to the parts of the industry-standard mobile risk list that touch the authentication surface:

Code	Risk	Covered by
M1	Improper Credential Usage	boolean-only auth, response oracle
M3	Insecure Authentication / Authorization (primary focus)	IPC oracle, response oracle, manifest
M6	Inadequate Privacy Controls	logcat leakage
M8	Security Misconfiguration	debuggable build
M9	Insecure Data Storage	allowBackup, logcat leakage
M10	Insufficient Cryptography	missing key-binding checks

7. Features left out or removed — and why

Timing side-channel testing (the "Mode A" on-device harness) REMOVED

What it was: a mode that measured, in nanoseconds, how long the app took to compare a secret, to detect "non-constant-time" code that leaks the secret through timing.

Why it was removed: on real consumer hardware the tiny timing signal (nanoseconds) was drowned out by ordinary phone background noise (scheduling, heat, other apps), so it only detected the planted flaw *some* of the time. It also required **rebuilding the target app** with special test code added — which breaks the whole "works on any app you don't control" promise. It was replaced by the **auth-state / response oracle** above, which tests a real side channel that is deterministic, needs no app rebuild, and fires every run.

White-box / instrumentation mode NOT INCLUDED

The tool is black-box only. It never needs the app's source code or a modified build. This keeps it usable against any app you are authorized to test and keeps the codebase simple.

Root-only inspection (reading `/data/data` directly) EXCLUDED

Directly opening another app's private files requires root. That was deliberately excluded so the tool runs on ordinary, unrooted phones — widening the audience who can use it.

Physical side channels (power / electromagnetic / acoustic / cache) OUT OF SCOPE

These require lab equipment or root-level access to hardware. They are outside what a pure software, no-root tool can honestly measure.

AI as a vulnerability *finder* DELIBERATELY EXCLUDED

The AI is never allowed to decide whether something is a vulnerability — that would make results unreliable and unrepeatable ("AI-washing"). Detection stays rule-based and deterministic; the AI only explains findings the rules already confirmed.

Machine learning in the statistics module DELIBERATELY EXCLUDED

At realistic test-run counts there simply isn't enough data to train a meaningful model, so the tool uses transparent, robust statistics instead of ML — honest about what the numbers can support.

OWASP M2, M4, M5, M7 OUT OF SCOPE

These cover supply chain, input validation, communication, and binary protections — outside the biometric-authentication surface this tool focuses on. Stating the boundary is a strength, not a gap.

8. Current limitations (stated honestly)

- **Scenario-driven checks are built but not yet active** FUTURE — the screen-capture / FLAG_SECURE check, the behavioural error-oracle, the lockout oracle, and the statistical baseline exist in the codebase but are not yet wired into a run. They need an "operator-assisted" loop that drives the app to the fingerprint prompt (a human presents a finger), which is planned future work.
- **Static findings are "likely", runtime findings are "confirmed".** Reading compiled code involves heuristics, so those findings are marked lower-confidence; anything proven by actually probing the live app is marked confirmed.
- **Packaged .exe start-up time.** The single-file Windows executable unpacks itself on each launch, so the first few seconds are start-up, not scanning. The scan itself is near-instant.

Only test applications you own or are explicitly authorized to test. BioAudit performs no exploitation beyond what is needed to demonstrate a finding, and every runtime action is gated behind an explicit authorization confirmation.